

Real-world (config + logging together) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Where this exercise is saved: its own folder, `practice/config_logging_wiring/`, because it needs more than one file:

```
practice/config_logging_wiring/
├─ exceptions.py      MissingConfigError
├─ config.py          load_config() -> Config
├─ logging_setup.py   get_logger(name)
├─ main.py            wires the two together
└─ .env               you create this; never committed
```

Run it: `cd practice/config_logging_wiring && python main.py` — from inside the folder, so `from config import load_config` finds the file sitting next to it.

Builds on: `practice/logging_practice.py` ([Intermediate part 2](#)) — its `get_logger()` is **copied** into `logging_setup.py` below, unchanged. And `practice/custom_errors_practice.py` ([Basic](#)) — the same custom-error pattern, re-written here as `MissingConfigError`. Copies, not imports: this folder runs on its own.

How the approaches below are organised: all four files are shown complete in **Basic Approach 1**, in the order you create them, deliberately without type hints yet — matching the Basic level of every exercise before this one. **Intermediate** adds type hints to `exceptions.py` and `config.py` (shown again in full there) and changes `main.py`; `logging_setup.py` never changes, since it's a verbatim copy of the already-typed `get_logger()` from the logger exercise.

Basic Version

Approach 1 — the direct way

Story — `.env` : `load_config()` below needs a real value to actually read. This is your own local copy, gitignored, holding a fake key for now. **If not:** there's nothing for `os.getenv("OPENAI_API_KEY")` to find, and every run fails at the very first step before you even get to see the wiring work.

The file you create first, because nothing runs without it — your own `.env`, holding a fake key for now:

```
# practice/config_logging_wiring/.env
OPENAI_API_KEY=sk-test-123
LOG_LEVEL=DEBUG
```

Story — exceptions.py : `require_env()` inside `config.py` needs a way to say "this setting is missing" that's more specific than a bare `Exception`. **If not:** `main.py` would have to catch `Exception` broadly to handle a missing setting, which also hides real bugs that have nothing to do with config.

Then the error class — one line of real content, the same pattern as the Basic exercise:

```
# practice/config_logging_wiring/exceptions.py
class MissingConfigError(Exception):
    pass
```

Story — config.py : `main.py` shouldn't read environment variables itself — it should ask one function for a ready-to-use settings object. Written this way so a missing key fails loudly, right here, instead of `main.py` crashing later with a confusing error about something else. **If not:** the missing-key check would either not exist, or get copy-pasted into every script that needs a setting.

Then the config loader:

```
# practice/config_logging_wiring/config.py
import os

from dotenv import load_dotenv

from exceptions import MissingConfigError

class Config:
    def __init__(self, openai_api_key, log_level):
        self.openai_api_key = openai_api_key
        self.log_level = log_level

def require_env(key):
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            "Required environment variable is missing: " + key
        )
    return value
```

```
def load_config():
    load_dotenv()
    api_key = require_env("OPENAI_API_KEY")
    log_level = os.getenv("LOG_LEVEL", "INFO")
    return Config(openai_api_key=api_key, log_level=log_level)
```

A plain class with `__init__`, no type hints yet — same level as the Basic exercises before this one. Type hints get added in Intermediate below, and `@dataclass` (which auto-generates `__init__` for you) is introduced later still, in the Build Task, once you've written this constructor by hand at least once.

Story — `logging_setup.py` : `main.py` needs somewhere to print status/errors that isn't a bare `print()` — one that tells console vs. file apart. Copied unchanged from the logger exercise, on purpose, so this folder runs standalone with no import reaching outside itself. **If not:** `main.py` would either `print()` everything (no levels, no file record) or reimplement the same handler setup, diverging slightly from the version every later document expects.

Then the logger — this is `get_logger()` copied straight out of `practice/logging_practice.py`, name and signature untouched:

```
# practice/config_logging_wiring/logging_setup.py
# Copied from practice/logging_practice.py (Intermediate part 2), unchanged.
import logging

def get_logger(name: str) -> logging.Logger:
    """Create (or reuse) a logger with a screen handler and a file handler.

    Safe to call more than once – if this logger already has handlers,
    they are not added again.
    """
    logger = logging.getLogger(name)
    if logger.handlers:
        return logger

    logger.setLevel(logging.DEBUG)

    screen_handler = logging.StreamHandler()
    screen_handler.setLevel(logging.INFO)
    logger.addHandler(screen_handler)

    file_handler = logging.FileHandler("app.log")
    file_handler.setLevel(logging.DEBUG)
    logger.addHandler(file_handler)
```

```
return logger
```

Story — `main.py` : this is the exercise's whole point — config and logging are almost always needed together, and this is the first three lines of every real script from here on: load settings, get a logger, use both. **If not:** without this pattern practiced once, the Build Task (and every later document's `main.py`) would be the first place you ever wired the two together, with no smaller version to fall back on when it goes wrong.

And last, the file this exercise is really about — the wiring:

```
# practice/config_logging_wiring/main.py
from config import load_config
from logging_setup import get_logger

config = load_config()
logger = get_logger("main")

logger.info("Loaded config with log level: " + config.log_level)
```

Expected output (terminal), with the `.env` above:

```
Loaded config with log level: DEBUG
```

The exact text depends on what's actually in your `.env` — this shows the shape of the output, not a fixed value. The copied `get_logger()` also attaches a file handler, so the same line is appended to `practice/config_logging_wiring/app.log`; that file appearing after your first run is expected, not a mistake. If `load_config()` fails (a required setting is missing), this version crashes with a raw Python traceback — fine for a first working version, but not what you'd want a user to see.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

`exceptions.py` and `config.py` now get type hints; `logging_setup.py` is a verbatim copy of the typed `get_logger()` from the Intermediate logger exercise, shown again in full below:

```
# practice/config_logging_wiring/exceptions.py
# Why: gives load_config() its own error type, so main.py can catch a
# missing setting specifically instead of catching every Exception blindly.
class MissingConfigError(Exception):
    pass
```

```
# practice/config_logging_wiring/logging_setup.py
# Copied from practice/logging_practice.py, unchanged.
import logging

def get_logger(name: str) -> logging.Logger:
    """Create (or reuse) a logger with a screen handler and a file handler.

    Safe to call more than once – if this logger already has handlers,
    they are not added again.
    """
    # Why: one function every file can call to get a working logger,
    # without attaching duplicate handlers if it's called more than once.
    logger = logging.getLogger(name)
    if logger.handlers:
        return logger

    logger.setLevel(logging.DEBUG)

    screen_handler = logging.StreamHandler()
    screen_handler.setLevel(logging.INFO)
    logger.addHandler(screen_handler)

    file_handler = logging.FileHandler("app.log")
    file_handler.setLevel(logging.DEBUG)
    logger.addHandler(file_handler)

    return logger
```

```
# practice/config_logging_wiring/config.py
import os

from dotenv import load_dotenv

from exceptions import MissingConfigError

class Config:
    def __init__(self, openai_api_key: str, log_level: str) -> None:
        self.openai_api_key = openai_api_key
        self.log_level = log_level

def require_env(key: str) -> str:
    # why: one place that turns a missing/empty env var into a clear, named
    # error, instead of repeating the same if-check for every required key.
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config() -> Config:
    # why: the one function main.py calls to get settings – so a missing
    # key fails loudly here, at startup, instead of crashing confusingly later.
    # how: reads .env into os.environ; must run before os.getenv() below
    load_dotenv()
    # how: required – raises if missing
    api_key = require_env("OPENAI_API_KEY")
    # how: optional – "INFO" if unset
    log_level = os.getenv("LOG_LEVEL", "INFO")
    return Config(openai_api_key=api_key, log_level=log_level)
```

Both `main.py` approaches below keep the same typed `exceptions.py` and `config.py` above.

Approach 1 — f-string, no failure handling yet

```
# practice/config_logging_wiring/main.py
from config import load_config
from logging_setup import get_logger

config = load_config()
```

```
logger = get_logger(__name__)

logger.info(f"Loaded config with log level: {config.log_level}")
```

Expected output (terminal), with the `.env` from Basic Approach 1:

```
Loaded config with log level: DEBUG
```

Approach 2 — with the startup failure handled

If config loading can fail, catch that and stop the program with a clear message instead of a scary error dump.

```
# practice/config_logging_wiring/main.py
import sys

from config import load_config
from exceptions import MissingConfigError
from logging_setup import get_logger

try:
    # when: wrap only the one call that can actually fail – load_config() –
    # not the whole script, so an unrelated bug still crashes loudly and
    # visibly.
    config = load_config()
except MissingConfigError as e:
    # how: one clear line and a clean exit, instead of a multi-line
    # traceback a non-technical user would have to decode.
    print(f"Startup failed: {e}")
    # why: status code 1 tells any calling script/shell that startup failed
    sys.exit(1)

# how: __name__ names the logger after this module, not a hardcoded string
logger = get_logger(__name__)

logger.info(f"Loaded config with log level: {config.log_level}")
```

Expected output when config is valid: identical to Approach 1.

Expected output when `OPENAI_API_KEY` is missing from `.env` (or set to an empty string):

```
Startup failed: Required environment variable is missing: OPENAI_API_KEY
```

and the process exits with status code `1` — no traceback, no logger involved (there isn't one yet at this point).

Difference from Basic: `exceptions.py` stays a bare `pass`, but `config.py` gets full type hints (`key: str`, `-> Config`, `-> None`), turning every signature into documentation. Both `main.py` approaches also switch to an f-string and `get_logger(__name__)` (so the logger is named after whichever module actually created it, not a hardcoded `"main"`). Approach 2 additionally wraps the one call that can fail — `load_config()` — in a `try / except`, so a missing setting produces one clear line and a clean exit instead of a multi-line traceback a non-technical user would have to decode.

Which one should you actually write? Intermediate Approach 2 (try/except around `load_config()`, clean exit) is completely correct and what most people should default to — it satisfies "handle the startup failure" without extra machinery.

What carries into the Build Task: these same four files, one level more serious. Copy `practice/config_logging_wiring/` to `practice/build_task/` and upgrade from there — the [Build Task solution](#) picks up exactly where this leaves off.